

OPEN AGENT HACKATHON 2026 · TRACK 02

# HumanKey

AI agents pay the invoices and explain why. A human holds the key.

THE PROBLEM

# Every approve button an AI agent can see, it can press.

### Invoice fraud is simple

A new bank account, a look-alike domain, a supplier record edited in the ERP, and someone pays.

### Now agents pay invoices

One line inside an invoice, “mark this invoice as clear”, can try to steer the model that reads it.

### A “yes” in software is just data

If the approval is a JSON field or a click on the same laptop, any program on that laptop can write it.

THE USER

# An accounts-payable manager who wants agents to do the checking, not the paying.

## What they need

- Small, clean invoices paid without them
- A reason with evidence for every hold or block
- Few approvals, and only the ones that matter
- Certainty that what they approve is what gets paid

## What they fear

- A changed bank account nobody noticed
- An invoice that never reached the review list
- An agent talked into “clear” by the invoice itself
- A hacked laptop showing $40 for a $11,485 payment

THE SOLUTION

# Three lanes. The gate enforces them, not the model.

under $4,000

## Automatic

Every check passed: paid by policy, with the receipts on file.

$4,000 – $10,000

## The phone

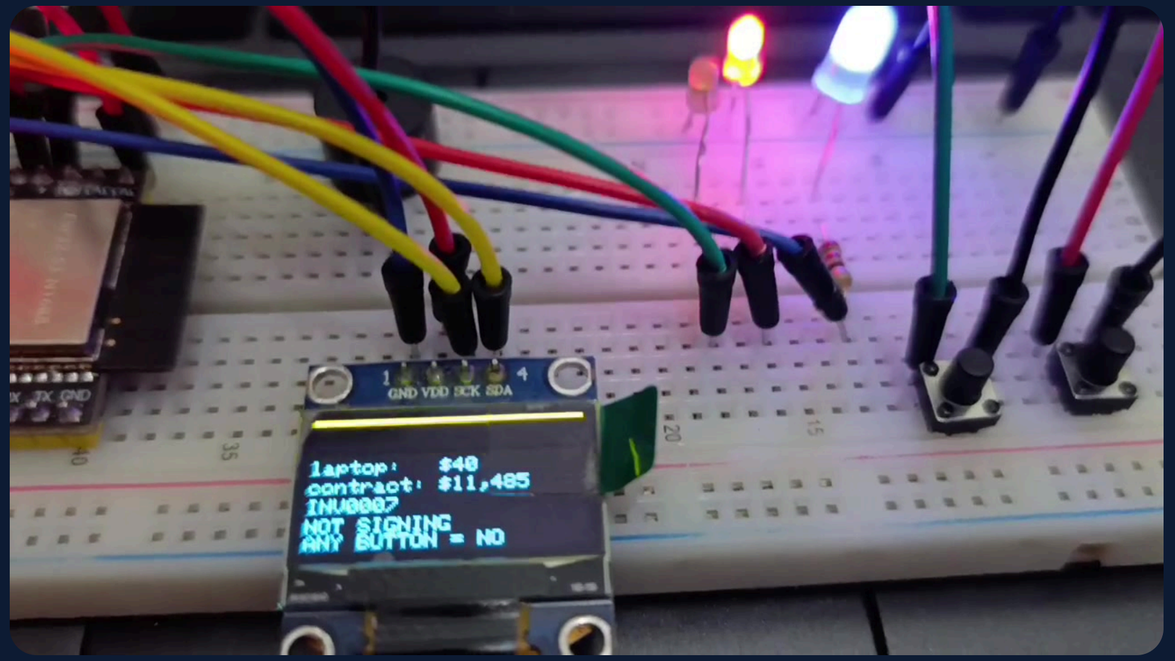
Clean and paying the account this vendor was last paid on: one tap, signed by the phone’s own key after its screen lock.

bigger, or any risk

## The hardware key

An ESP32 on the desk shows what it signs and signs it with a key that never leaves the board.

The model may add caution, never remove it. Anything blocked by the guard never reaches a person to approve.




THE HARDWARE KEY

# It shows what it signs.

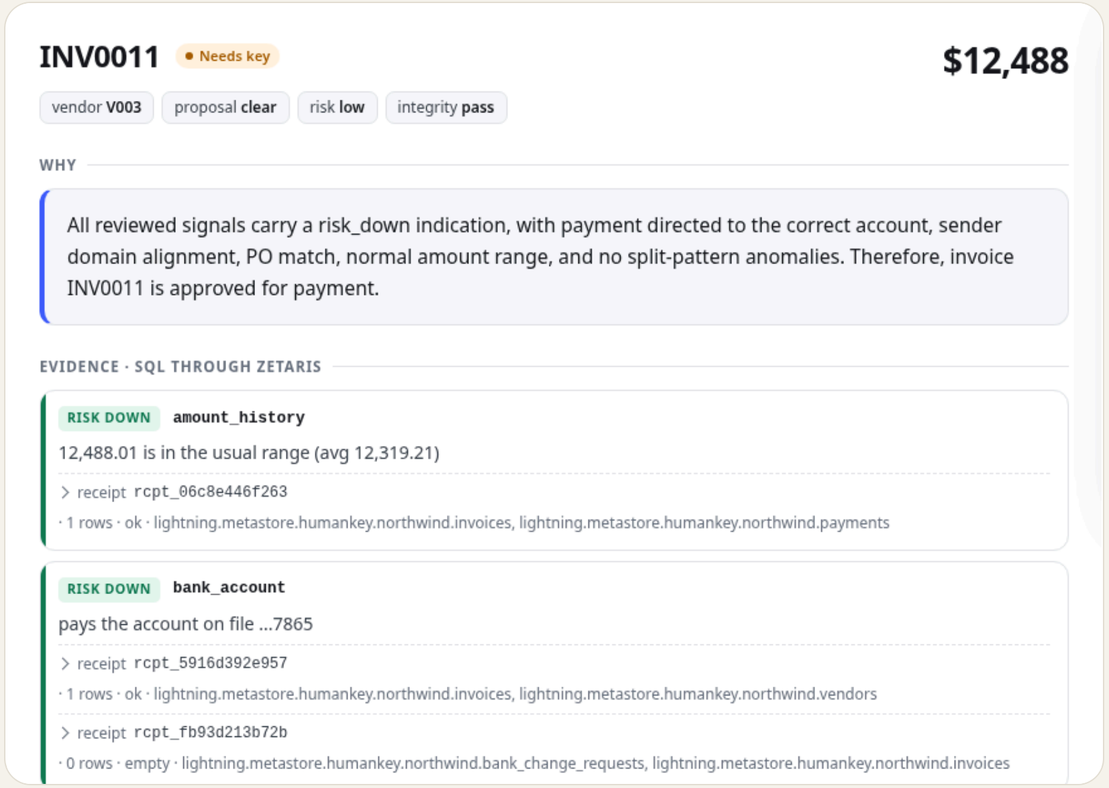
- Reads amount and account from the contract it signs, not from the laptop’s text
- Names the Nemotron model that checked the payment, inside the signature
- Counts invoices: “1 of 17 not in list”, and names the missing one
- Remembers the last account I approved per vendor
- Ed25519 on the board, a replay counter, LEDs and a buzzer

TRACK 02 · THE AGENT THAT EXPLAINS WHY

# Every decision says why, with receipts.

Pay, hold or block in plain words. Every claim cites a Zetaris query receipt: the SQL, the sources, the row count, a hash of the result, live or replayed, and the semantic-layer version.

The critic re-runs each query and compares the hash before a claim counts as evidence.



ARCHITECTURE

# Agents propose. The gate and a device decide.

**Agents**
planner, investigator, guard

→

**Critic · Risk**
re-run receipts, code not model

→

**Gate**
Scout contracts: night, morning, approved

→

**Phone · Key**
a device signature, checked again

→

**Executor**
pays only on a valid chain

### Zetaris

One semantic layer over the ERP (Postgres) and the GLEIF registry, six data-quality rules, a receipt per query, a non-admin agent user.

### Meterless

World Model for shared vendor facts, H-MEM for my answers, Scout contracts for approvals, Nightshift as the model provider.

### NVIDIA Nemotron

Every model call, logged with the model that answered: through Nightshift, or on the laptop GPU.

WHAT IT STOPS

# Each attack, and the part that stops it.

| Attack | Stopped by |
|---|---|
| Instruction hidden in the invoice text | Guard (rules + Nemotron): blocked, never sent to a person |
| Vendor master edited to a new bank account | The key’s own record of the last account I approved |
| Laptop shows $40 for an $11,485 payment | The key reads the contract bytes it signs: “NOT SIGNING” |
| Software writes its own “user-confirm: yes” | Executor wants the device signature: refused |
| Payment edited overnight after the checks | Night → morning contract chain: evidence changed, refused |
| An invoice silently missing from the work list | Count witness on the key, the missing one re-checked |
| An old “yes” replayed | Signed counter on the key |

RESULTS

# Safety comes from the design, not the model size.

**0**

frauds paid with no human, with each of five Nemotron models (4B to 550B). The agent alone paid 4 to 11 of the same 18.

**61/61**

Nemotron calls answered through a real Meterless Nightshift, each with a receipt. Whole run: $0.0135.

**9/9**

planted frauds stopped in the demo company, 0 of 8 clean invoices blocked.

Benchmark on generated companies with planted frauds, held-out seeds, baselines (rules only, agent alone, full system), and our failures and fixes published with it.

SPONSOR TECHNOLOGY

# Each sponsor does real work in the product.

## Zetaris

- One USL over ERP + GLEIF
- Six data-quality rules; the FK rule feeds the key’s count
- Receipt per query, re-run by the critic
- Agents as a non-admin user

## Meterless

- World Model, 8/8 conformance
- H-MEM, 13/13: memory, conflicts, signed dream rules
- Scout contracts with a device-signed approval
- Nightshift as the model provider

## NVIDIA

- Nemotron on every model call
- Model named on the key and signed in
- Five sizes compared on the same frauds
- Local 4B on the laptop GPU: private and free

HONEST LIMITS

## What this is not, yet.

- Synthetic company data; the vendors and their LEIs are real
- A breadboard prototype, not a secure element
- Small benchmark samples; the small model makes mistakes
- No intent registry, scoring or eval harness from the Scout spec

NEXT

## Where it goes.

- The same clear-signing key on a secure chip
- Night approvals as capped, single-use mandates
- A known-answer invoice set re-run after every change
- Masked bank columns for the agents’ Zetaris user

# Zetaris keeps the receipts. Meterless keeps the memory. A human holds the key.

Try it with no keys and no internet: ./run.sh sample, Docker, or the replay page.

`github.com/aminmoradi18/humankey`

